HOMEWORK – 5

1.Use linked list for polynomial representation and perform addition and subtraction operations on the polynomials.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff;
   int exp;
   struct Node *next;
};

struct Node* createNode(int coeff, int exp) {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->coeff = coeff;
   newNode->exp = exp;
   newNode->next = NULL;

   return newNode;
}

struct Node* insert(struct Node *head, int coeff, int exp) {
   struct Node *newNode = createNode(coeff, exp);
   struct Node *temp;
```

```
    if (head == NULL)
        return newNode;

    temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    return head;
}

void display(struct Node *head) {
    struct Node *temp = head;

    while (temp != NULL) {
        printf("%dx^%d", temp->coeff, temp->exp);

        if (temp->next != NULL)
            printf(" + ");

        temp = temp->next;
    }

    printf("\n");
}
```

```
struct Node* addPolynomial(struct Node *p1, struct Node *p2) {
  struct Node *result = NULL;

  while (p1 != NULL && p2 != NULL) {

    if (p1->exp == p2->exp) {
      result = insert(result,
                p1->coeff + p2->coeff,
                p1->exp);

      p1 = p1->next;
      p2 = p2->next;
    }
    else if (p1->exp > p2->exp) {
      result = insert(result, p1->coeff, p1->exp);
      p1 = p1->next;
    }
    else {
      result = insert(result, p2->coeff, p2->exp);
      p2 = p2->next;
    }
  }

  while (p1 != NULL) {
    result = insert(result, p1->coeff, p1->exp);
    p1 = p1->next;
  }
```

```
    while (p2 != NULL) {
        result = insert(result, p2->coeff, p2->exp);
        p2 = p2->next;
    }

    return result;
}

int main() {
    struct Node *p1 = NULL, *p2 = NULL, *sum = NULL;
    int n, coeff, exp, i;

    printf("Enter number of terms in polynomial 1: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &coeff, &exp);
        p1 = insert(p1, coeff, exp);
    }

    printf("Enter number of terms in polynomial 2: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &coeff, &exp);
        p2 = insert(p2, coeff, exp);
```

```
    }

    printf("\nPolynomial 1: ");
    display(p1);

    printf("Polynomial 2: ");
    display(p2);

    sum = addPolynomial(p1, p2);

    printf("Sum: ");
    display(sum);

    return 0;
}
```

OUTPUT:

```
Enter number of terms in polynomial 2: 2
Enter coefficient and exponent: 3 1
Enter coefficient and exponent: 6 0

Polynomial 1: 2x^2 + 7x^0
Polynomial 2: 3x^1 + 6x^0
Sum: 2x^2 + 3x^1 + 13x^0
```

2.Find previous node of circular linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *head = NULL, *newNode, *temp;
   int n, i, value, key;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {

      newNode = (struct Node*)malloc(sizeof(struct Node));

      printf("Enter data: ");
      scanf("%d", &newNode->data);

      newNode->next = NULL;

      if (head == NULL) {
         head = newNode;
         newNode->next = head;
      }
      else {
         temp = head;
```

```
      while (temp->next != head)
         temp = temp->next;

      temp->next = newNode;
      newNode->next = head;
   }
}

printf("Enter value whose previous node is required: ");
scanf("%d", &key);

temp = head;

do {
   if (temp->next->data == key) {
      printf("Previous node of %d is %d\n",
           key, temp->data);
      return 0;
   }

   temp = temp->next;

} while (temp != head);

printf("Element not found\n");

return 0;
```

```
}
```

OUTPUT:

```
Enter number of nodes: 5
Enter data: 12
Enter data: 34
Enter data: 56
Enter data: 78
Enter data: 90
Enter value whose previous node is required: 78
```

3.Insert at a given position of a circular linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *head = NULL, *newNode, *temp;
   int n, i, pos, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
```

```
    newNode = (struct Node*)malloc(sizeof(struct Node));

    printf("Enter data: ");
    scanf("%d", &newNode->data);

    if (head == NULL) {
        head = newNode;
        newNode->next = head;
    }
    else {
        temp = head;

        while (temp->next != head)
            temp = temp->next;

        temp->next = newNode;
        newNode->next = head;
    }
}

printf("Enter position: ");
scanf("%d", &pos);

printf("Enter value: ");
scanf("%d", &value);

newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
newNode->data = value;

if (pos == 1) {

   temp = head;

   while (temp->next != head)
      temp = temp->next;

   newNode->next = head;
   temp->next = newNode;
   head = newNode;
}

else {

   temp = head;

   for (i = 1; i < pos - 1; i++)
      temp = temp->next;

   newNode->next = temp->next;
   temp->next = newNode;
}

printf("Circular Linked List:\n");

temp = head;
```

```
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    return 0;
}
```

OUTPUT:

```
Enter number of nodes: 5
Enter data: 12
Enter data: 34
Enter data: 56
Enter data: 78
Enter data: 90
Enter position: 3
Enter value: 40
Circular Linked List:
12 34 40 56 78 90
```

4. Find largest second element in a Queue.

CODE:

```
#include <stdio.h>

int main() {
    int queue[100];
    int n, i;
    int largest, second;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter queue elements:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &queue[i]);

    largest = second = -999999;

    for (i = 0; i < n; i++) {

        if (queue[i] > largest) {
            second = largest;
            largest = queue[i];
        }
        else if (queue[i] > second && queue[i] != largest) {
            second = queue[i];
        }
    }

    printf("Largest element = %d\n", largest);
    printf("Second largest element = %d\n", second);

    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter queue elements:

12 34 45 67 9

Largest element = 67

Second largest element = 45

5.Insert 3rd element and check whether 1st and 3rd elements are same in a linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *head = NULL, *newNode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {

      newNode = (struct Node*)malloc(sizeof(struct Node));

      printf("Enter data: ");
```

```
    scanf("%d", &newNode->data);

    newNode->next = NULL;

    if (head == NULL) {
      head = newNode;
    }
    else {
      temp = head;

      while (temp->next != NULL)
        temp = temp->next;

      temp->next = newNode;
    }
}

printf("Enter element to insert at 3rd position: ");
scanf("%d", &value);

newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;

temp = head;

for (i = 1; i < 3 - 1; i++)
    temp = temp->next;
```

```
newNode->next = temp->next;
temp->next = newNode;

printf("Updated Linked List:\n");

temp = head;

while (temp != NULL) {
   printf("%d ", temp->data);
   temp = temp->next;
}

printf("\n");

temp = head;

int first = temp->data;

temp = temp->next;
temp = temp->next;

int third = temp->data;

if (first == third)
   printf("1st and 3rd elements are SAME\n");
else
   printf("1st and 3rd elements are NOT SAME\n");
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of nodes: 5
Enter data: 1
Enter data: 3
Enter data: 1
Enter data: 4
Enter data: 5
Enter element to insert at 3rd position: 1
Updated Linked List:
1 3 1 1 4 5
1st and 3rd elements are SAME
```

6. Implement two stacks in one array. Write routines to traverse the array and display.

CODE:

```
#include <stdio.h>
#define MAX 10

int stack[MAX];
int top1 = -1;
int top2 = MAX;

void push1(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
        return;
```

```
    }

    top1++;
    stack[top1] = value;
}

void push2(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
        return;
    }

    top2--;
    stack[top2] = value;
}

void pop1() {
    if (top1 == -1) {
        printf("Stack 1 Underflow\n");
        return;
    }

    printf("Deleted: %d\n", stack[top1]);
    top1--;
}

void pop2() {
    if (top2 == MAX) {
```

```
        printf("Stack 2 Underflow\n");
        return;
    }

    printf("Deleted: %d\n", stack[top2]);
    top2++;
}

void display() {
    int i;

    printf("Stack 1: ");

    for (i = 0; i <= top1; i++)
        printf("%d ", stack[i]);

    printf("\n");

    printf("Stack 2: ");

    for (i = MAX - 1; i >= top2; i--)
        printf("%d ", stack[i]);

    printf("\n");
}

int main() {
```

```
    push1(10);
    push1(20);
    push1(30);

    push2(90);
    push2(80);
    push2(70);

    display();

    pop1();
    pop2();

    display();

    return 0;
}
```

OUTPUT:

```
Stack 1: 10 20 30
Stack 2: 90 80 70
Deleted: 30
Deleted: 70
Stack 1: 10 20
Stack 2: 90 80
```

7. Each node has two pointers: The head nodes are sorted, and each sub-list is also sorted. We have to **f**latten everything into one sorted linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int data;
   struct Node *next;
   struct Node *bottom;
};

struct Node* createNode(int data)
{
   struct Node *newNode;

   newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->next = NULL;
   newNode->bottom = NULL;

   return newNode;
}
struct Node* merge(struct Node *a, struct Node *b)
{
   struct Node *result;

   if (a == NULL)
      return b;
```

```
    if (b == NULL)
        return a;

    if (a->data <= b->data)
    {
        result = a;
        result->bottom = merge(a->bottom, b);
    }
    else
    {
        result = b;
        result->bottom = merge(a, b->bottom);
    }

    return result;
}
struct Node* flatten(struct Node *head)
{
    if (head == NULL || head->next == NULL)
        return head;
    head->next = flatten(head->next);
    head = merge(head, head->next);

    return head;
}

void display(struct Node *head)
```

```
{
    while (head != NULL)
    {
        printf("%d", head->data);

        if (head->bottom != NULL)
            printf(" -> ");

        head = head->bottom;
    }
}

int main()
{
    struct Node *head;
    head = createNode(5);
    head->next = createNode(10);
    head->next->next = createNode(19);
    head->next->next->next = createNode(28);
    head->bottom = createNode(7);
    head->bottom->bottom = createNode(8);
    head->next->bottom = createNode(20);
    head->next->next->bottom = createNode(22);
    head->next->next->next->bottom = createNode(40);
    head->next->next->next->bottom->bottom = createNode(45);

    head = flatten(head);
```

```
    printf("Flattened sorted list:\n");
    display(head);

    return 0;
}
```

OUTPUT:

```
Flattened sorted list:
5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45
```

8. Given k sorted linked list of different sizes, we need to merge them into a single list maintaining their sorted order.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int data)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
```

```
    return newNode;
}

struct Node* insert(struct Node *head, int data)
{
    struct Node *newNode, *temp;

    newNode = createNode(data);

    if (head == NULL)
        return newNode;

    temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    return head;
}

struct Node* merge(struct Node *head1, struct Node *head2)
{
    struct Node *result = NULL;

    while (head1 != NULL && head2 != NULL)
    {
```

```
        if (head1->data < head2->data)
        {
            result = insert(result, head1->data);
            head1 = head1->next;
        }
        else
        {
            result = insert(result, head2->data);
            head2 = head2->next;
        }
    }

    while (head1 != NULL)
    {
        result = insert(result, head1->data);
        head1 = head1->next;
    }

    while (head2 != NULL)
    {
        result = insert(result, head2->data);
        head2 = head2->next;
    }

    return result;
}

void display(struct Node *head)
```

```
{
   while (head != NULL)
   {
      printf("%d ", head->data);
      head = head->next;
   }
}

int main()
{
   struct Node *head1 = NULL;
   struct Node *head2 = NULL;
   struct Node *result;

   int n, i, data;

   printf("Enter number of nodes in first list: ");
   scanf("%d", &n);

   printf("Enter sorted elements:\n");

   for (i = 0; i < n; i++)
   {
      scanf("%d", &data);
      head1 = insert(head1, data);
   }

   printf("Enter number of nodes in second list: ");
```

```
    scanf("%d", &n);

    printf("Enter sorted elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &data);
        head2 = insert(head2, data);
    }

    result = merge(head1, head2);

    printf("Merged sorted list:\n");
    display(result);

    return 0;
}
```

OUTPUT:

```
Enter number of nodes in first list: 3
Enter sorted elements:
1 5 8
Enter number of nodes in second list: 3
Enter sorted elements:
2 4 7
Merged sorted list:
1 2 4 5 7 8
```

9. Swap kth node from beginning with kth node from end in a linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node* createNode(int data)
{
   struct Node *newNode;

   newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->next = NULL;

   return newNode;
}

struct Node* insert(struct Node *head, int data)
{
   struct Node *newNode, *temp;

   newNode = createNode(data);

   if (head == NULL)
```

```
        return newNode;

    temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    return head;
}

void swapKth(struct Node **head, int k)
{
    struct Node *x, *y, *temp;
    struct Node *prevX = NULL, *prevY = NULL;
    int n = 0, i;

    temp = *head;

    while (temp != NULL)
    {
        n++;
        temp = temp->next;
    }
    if (k > n || k <= 0)
    {
        printf("Invalid k\n");
```

```
    return;
}

if (2 * k - 1 == n)
    return;

x = *head;

for (i = 1; i < k; i++)
{
    prevX = x;
    x = x->next;
}
y = *head;

for (i = 1; i < n - k + 1; i++)
{
    prevY = y;
    y = y->next;
}

if (prevX != NULL)
    prevX->next = y;
else
    *head = y;

if (prevY != NULL)
    prevY->next = x;
```

```
    else
        *head = x;

    if (x->next == y)
    {
        x->next = y->next;
        y->next = x;
    }
    else if (y->next == x)
    {
        y->next = x->next;
        x->next = y;
    }
    else
    {
        temp = x->next;
        x->next = y->next;
        y->next = temp;
    }
}

void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->next;
    }
```

```
}

int main()
{
   struct Node *head = NULL;
   int n, i, data, k;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter elements:\n");

   for (i = 0; i < n; i++)
   {
      scanf("%d", &data);
      head = insert(head, data);
   }

   printf("Enter k: ");
   scanf("%d", &k);

   swapKth(&head, k);

   printf("After swapping:\n");
   display(head);

   return 0;
}
```

OUTPUT:

```
Enter number of nodes: 6
Enter elements:
12 34 56 78 90 10
Enter k: 3
After swapping:
12 34 78 56 90 10
```

10. Multiply two numbers represented by two linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node* createNode(int data)
{
   struct Node *newNode;

   newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;

   return newNode;
}
```

```
struct Node* insert(struct Node *head, int data)
{
   struct Node *newNode, *temp;

   newNode = createNode(data);

   if (head == NULL)
      return newNode;

   temp = head;

   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;

   return head;
}

long long convert(struct Node *head)
{
   long long num = 0;

   while (head != NULL)
   {
      num = num * 10 + head->data;
      head = head->next;
```

```
    }

    return num;
}

int main()
{
    struct Node *head1 = NULL;
    struct Node *head2 = NULL;

    int n, i, digit;
    long long num1, num2, result;

    printf("Enter number of digits in first number: ");
    scanf("%d", &n);

    printf("Enter digits:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &digit);
        head1 = insert(head1, digit);
    }

    printf("Enter number of digits in second number: ");
    scanf("%d", &n);

    printf("Enter digits:\n");
```

```
    for (i = 0; i < n; i++)
    {
        scanf("%d", &digit);
        head2 = insert(head2, digit);
    }

    num1 = convert(head1);
    num2 = convert(head2);

    result = num1 * num2;

    printf("Product = %lld\n", result);

    return 0;
}
```

OUTPUT:

```
Enter number of digits in first number: 3
Enter digits:
1 2 3
Enter number of digits in second number: 2
Enter digits:
2 3
Product = 2829
```